

Desafío I: Sweet Crush

Informe preliminar: Contextualización, Análisis y Diseño
Integrante: Sergio Cabrera (1086922688) – Samuel Bastidas (1138525766)

1.Contextualización

El Desafío I plantea diseñar e implementar el juego Sweet Crush bajo una restricción que define por completo el resto del proyecto: ninguna posición del tablero puede representarse con un tipo de dato nativo completo, como un entero o un carácter. En su lugar, cada ficha debe empaquetarse en 3 bits dentro de una secuencia continua de memoria dinámica, sin desperdiciar espacio en alineaciones a byte. Esta decisión de diseño obliga a resolver simultáneamente dos problemas de diferentes: por un lado, la manipulación precisa de información a nivel de bits mediante los operadores `&`, `|`, `^`, `~`, `<<` y `>>`, necesarios para localizar, extraer, construir y modificar el valor de una ficha sin desempaquetar jamás el tablero en una estructura auxiliar de bytes o enteros; por otro lado, la administración responsable de memoria dinámica bajo el framework Qt, reservando y liberando exactamente la cantidad de memoria que corresponde a las dimensiones vigentes del tablero en cada momento, sin sobredimensionar la estructura ni anticipar espacio para expansiones futuras.

El desarrollo del proyecto además tiene restricciones de lenguaje que son el uso de `struct`, `class`, `template`, `std::string` y cualquier contenedor de la biblioteca estándar de plantillas, así como cualquier sintaxis de ANSI C.

2.Análisis del problema

2.1 Requerimientos funcionales identificados

El programa debe permitir crear un tablero rectangular cuyas dimensiones se indiquen al iniciar la ejecución, y mostrarlo de una manera comprensible para el jugador en dos formatos distintos, uno simbólico, con caracteres representativos de cada tipo de ficha, y otro binario, mostrando el contenido crudo de la memoria, ambos desplegados después de cada jugada. Durante la partida, el jugador selecciona una posición del tablero para eliminar la ficha allí ubicada, y el sistema debe determinar automáticamente si dicha eliminación produjo una combinación válida, entendida como tres o más fichas iguales ubicadas consecutivamente en sentido horizontal o vertical. Es indispensable que una misma ficha pueda participar simultáneamente en una combinación horizontal y en una vertical sin que el procesamiento de una invalide el de la otra, lo cual exige un mecanismo de detección que marque las coincidencias antes de eliminarlas físicamente.

Tras una eliminación, las posiciones vacías deben gestionarse mediante un desplazamiento vertical de las fichas restantes hacia abajo, seguido de la incorporación de fichas nuevas generadas aleatoriamente bajo una distribución uniforme, de manera que el tablero recupere sus dimensiones originales. Dado que este relleno puede generar combinaciones adicionales sin intervención del jugador, el sistema debe detectarlas automáticamente y procesarlas en un ciclo que se denomina cascada, repitiéndose hasta que el tablero alcance un estado sin combinaciones nuevas. Adicionalmente, el programa debe permitir agregar o eliminar una fila o una columna en cualquier posición válida de la estructura, incluidas posiciones intermedias y no únicamente los extremos, reorganizando el tablero para que permanezca completamente funcional después de cada modificación. Finalmente, el sistema debe mantener un registro del estado de la partida y aplicar un criterio de puntuación.

2.2 Restricciones técnicas que condicionan el diseño

Aparte de la codificación de 3 bits por ficha, el enunciado exige que la memoria reservada corresponda en todo momento a las dimensiones actuales del tablero, prohibiendo explícitamente mantener dimensiones máximas de forma permanente o reservar de antemano espacio para expansiones futuras. También establece una regla sobre el orden interno de los bits, la trama de bits válidos debe iniciar siempre en el bit menos significativo del primer byte, y si el total de bits requeridos no es múltiplo de ocho, los bits sobrantes deben agruparse como los bits más significativos del último byte del arreglo. En cuanto a la reducción de memoria tras eliminar filas o columnas, se exige que la reasignación física solo ocurra cuando la ocupación real caiga por debajo del 65% por ciento de la capacidad actualmente reservada, evitando el costo de realojar memoria en cada eliminación individual.

3. Diseño de la solución

3.1 Codificación de las fichas

Con tres bits por posición se dispone de ocho combinaciones posibles, de las cuales el enunciado exige reservar seis para representar los seis tipos de ficha jugables del juego. Asignamos los códigos binarios entre 000 y 101 a los seis tipos, dejando libre la asociación concreta entre cada código y su ficha correspondiente, tal como dice el enunciado. De las dos combinaciones restantes, el código 110 se reserva para representar una posición vacía, es decir, el estado que adopta una casilla inmediatamente después de que su ficha ha sido eliminada de forma definitiva. El código 111, se reserva exclusivamente como marca temporal utilizada durante el algoritmo de detección de combinaciones en dos pasadas, a propósito no se le asigna ningún otro significado, ya que introducir una segunda función para ese mismo código generaría un error entre una posición realmente marcada para eliminación y una ficha especial presente en el tablero.

3.2 Representación física y bits

El tablero se almacena en un bloque de memoria dinámica de tipo unsigned char*, donde cada byte no corresponde a una ficha individual sino a un fragmento de la secuencia continua de bits. Para ubicar una ficha dada su fila y columna, primero se calcula su índice lógico lineal multiplicando la fila por la cantidad de columnas y sumando la columna; ese índice se multiplica por tres para obtener el bit inicial global de la ficha dentro de la secuencia completa; la posición del byte físico que la contiene se obtiene dividiendo ese bit inicial entre ocho, y el desplazamiento interno dentro de ese byte se obtiene con el residuo de dicha división. Siguiendo la restricción sobre el orden de los bits, el bit cero de esta numeración corresponde siempre al bit menos significativo del primer byte del arreglo, y las fichas se empaquetan de forma continua sin dejar huecos entre una y otra; como consecuencia directa, cuando la cantidad total de bits no es múltiplo de ocho, los bits sobrantes quedan automáticamente ubicados como los bits más significativos del último byte.

La operación de extracción o escritura de una ficha se resuelve según tres casos posibles, determinados por el desplazamiento interno. Si dicho desplazamiento es menor o igual a cinco, la ficha completa cabe dentro de un único byte, ya que desplazamiento más tres no supera ocho, y basta con desplazar el byte hacia la derecha la cantidad de posiciones indicada y aplicar una máscara de tres bits para aislar su valor. Si el desplazamiento es igual a seis, únicamente dos de los tres bits de la ficha caben en el byte actual, mientras que el bit restante ocupa la posición menos significativa del byte siguiente; el valor se reconstruye tomando esos dos bits del byte actual y combinándolos mediante un operador lógico OR con el bit menos significativo del byte siguiente, previamente desplazado dos posiciones hacia la izquierda para ocupar el lugar más significativo del valor final. Si el desplazamiento es igual a siete, únicamente un bit de la ficha permanece en el byte actual, en su posición más significativa, mientras que los dos bits restantes ocupan las posiciones menos significativas del byte siguiente; ambos fragmentos se combinan de manera igual al caso anterior. La escritura de una ficha sigue el mismo particionamiento en sentido inverso: primero se limpia la posición destino aplicando una máscara invertida mediante los operadores NOT y AND, y posteriormente se inyecta el nuevo valor mediante el operador OR, repitiendo el procedimiento en el byte siguiente cuando corresponda.

3.3 Organización modular del proyecto

La solución se dividirá en cinco unidades de compilación independientes para respetar la exigencia de programación modular multi archivo. Un primer módulo, dedicado exclusivamente a bits y memoria, concentrará el cálculo de índices, la extracción y escritura de una ficha y el cómputo de la cantidad mínima de bytes necesarios mediante redondeo hacia arriba. Un segundo módulo, dedicado a la gestión del tablero, se encargará de su creación inicial, de la inserción y eliminación de filas y columnas en cualquier posición y del redimensionamiento real de la memoria, incluyendo la aplicación de la regla del 65% al reducir dimensiones. Un tercer módulo concentrará la lógica de juego y las cascadas, incluyendo el llenado aleatorio inicial, la detección de combinaciones, la eliminación de fichas marcadas, la aplicación de gravedad y relleno, y el control del ciclo de cascadas junto con la puntuación. Un cuarto módulo se encargará exclusivamente de la interfaz por consola, imprimiendo el tablero tanto en su representación simbólica como en su representación binaria. Finalmente, el main contendrá el bucle de la partida y el menú interactivo mediante el cual el jugador realiza sus jugadas y solicita modificaciones estructurales, coordinando el trabajo de los demás módulos sin implementar lógica propia adicional.

3.4 Algoritmos

La detección de combinaciones se resolverá mediante un algoritmo de dos pasadas sobre el tablero. En la primera pasada se recorre la totalidad de las posiciones, tanto por filas como por columnas, identificando rachas de tres o más fichas consecutivas del mismo tipo; toda ficha que forme parte de alguna de estas rachas se reemplaza temporalmente por el código 111 en lugar de eliminarse de inmediato, de manera que una ficha que participe simultáneamente en una combinación horizontal y en una vertical sea marcada una sola vez sin que la detección de una dirección interfiera con la otra. En la segunda pasada se recorre nuevamente la memoria completa y toda posición marcada con el código 111 se reescribe al código 110, quedando así vacía de manera permanente. Inmediatamente después se ejecuta el algoritmo de gravedad, que recorre cada columna de abajo hacia arriba; al encontrar una posición vacía, busca la ficha válida más próxima por encima de ella, transfiere sus bits hacia la posición inferior y deja vacía la posición de origen, para luego rellenar los huecos que permanezcan en la parte superior de la columna con fichas nuevas generadas aleatoriamente bajo una distribución uniforme sobre los seis tipos disponibles.

El redimensionamiento de la memoria seguirá un criterio distinto según se trate de una ampliación o de una reducción. Al insertar una fila o una columna, se reservará un nuevo bloque de memoria con el tamaño exacto que corresponde a las nuevas dimensiones, se copiarán las fichas existentes preservando su orden lógico, se rellenará la nueva fila o columna con fichas aleatorias, y se recompactará la totalidad del contenido comenzando siempre desde el bit cero del nuevo bloque, liberando el bloque anterior una vez concluida la copia. Cuando la posición de inserción sea intermedia, las filas o columnas posteriores a su posición se desplazarán en bloque para dejar espacio a la nueva; en el caso particular de una columna, esto implica reconstruir el tablero fila por fila, dado que el ancho de cada fila cambia. La eliminación de una fila o columna sigue un procedimiento equivalente en sentido inverso, omitiendo durante la copia la fila o columna indicada y recompactando el resto, sin embargo, la reducción física del bloque de memoria solamente se ejecutará cuando, tras la eliminación, la cantidad de memoria utilizada caiga por debajo del 65% de la capacidad actualmente reservada, evitando así realojar memoria de forma innecesaria ante cada eliminación individual. En todos los casos, tanto una eliminación manual como una modificación estructural o la incorporación de fichas nuevas mediante el relleno aleatorio deben disparar nuevamente el ciclo de detección de combinaciones, repitiéndose mientras sigan apareciendo coincidencias nuevas y registrando cada repetición como una cascada adicional.

3.5 Estado del juego y puntuación

El programa mantendrá en todo momento seis variables de estado exigidas por el enunciado: las dimensiones actuales del tablero, la cantidad de eliminaciones realizadas directamente por el jugador, la cantidad total de fichas eliminadas a lo largo de la partida, la cantidad de combinaciones detectadas, la cantidad de cascadas producidas por la jugada en curso y la puntuación acumulada. Como criterio de puntuación, se propone otorgar diez puntos por cada ficha eliminada dentro de una combinación.

3.6 Dificultades

Pensamos en varias dificultades. La primera consiste en la precisión requerida para el manejo de bits entre bytes consecutivos al escribir una ficha repartida entre dos de ellos, dado que una máscara construida incorrectamente podría corromper bits pertenecientes a fichas vecinas. La segunda se relaciona con la recompactación del tablero tras insertar o eliminar una fila o columna en una posición intermedia, operación que puede desplazar en bloque una cantidad considerable de fichas y que exige un control cuidadoso de los índices durante la copia. La tercera dificultad anticipada es el manejo manual de memoria dinámica: cada reserva realizada debe corresponder exactamente a una liberación, en particular durante las reasignaciones asociadas a la regla del 65%, donde un error de sincronización entre el puntero antiguo y el nuevo podría producir fugas de memoria o punteros inválidos.

3.7 Entorno de desarrollo

El proyecto se desarrollará en C++ dentro de Qt Creator, sin necesidad de recurrir a una interfaz gráfica, dado que la visualización del tablero se realizará exclusivamente por consola. Toda la entrada y salida del programa se implementará mediante cin y cout, evitando por completo ANSI C, en su lugar, toda reserva y liberación de memoria dinámica se realizará exclusivamente mediante los operadores new y delete.